O'REILLY®
Report

# Managing Memory for AI Agents

## Memory, Models, and the Architecture of Adaptability

Benjamin Labaschin, Jim Allen Wallace, Andrew Brookins & Manvinder Singh

Compliments of Redis

# IT'S NOT YOUR MODEL. IT'S YOUR MEMORY.

AI agents need memory.
Redis has it. redis.io/redis-for-ai

Redis

# Managing Memory for AI Agents

*Memory, Models, and the Architecture of Adaptability*

*Benjamin Labaschin, Jim Allen Wallace, Andrew Brookins, and Manvinder Singh*

O'REILLY®

**Managing Memory for AI Agents**

by Benjamin Labaschin, Jim Allen Wallace, Andrew Brookins, and Manvinder Singh

Copyright © 2025 O'Reilly Media, Inc. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*https://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

**Acquisitions Editor:** Aaron Black
**Development Editor:** Jill Leonard
**Production Editor:** Jonathon Owen
**Copyeditor:** Shannon Turlington
**Cover Designer:** Susan Thompson
**Cover Illustrator:** Ellie Volckhausen
**Interior Designer:** David Futato
**Interior Illustrator:** Kate Dullea

October 2025: First Edition

**Revision History for the First Edition**

2025-10-03: First Release

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Managing Memory for AI Agents*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and Redis. See our statement of editorial independence.

979-8-341-66122-6

[LSI]

# Table of Contents

**Introduction** . . . . . v

**1. A Deep Dive into Agent Memory Systems** . . . . . 1
Understanding Agent Memory Systems 2
Memory Storage and Representation 3
The Challenge of Nondeterministic Systems 3
Storage and Retrieval: The Core Challenge 4
The Imprecision of Retrieval 5
Managing Context Window Limitations 5
Persistence via Checkpointing 6

**2. Long-Term Memory: Building Persistent Learning Agents** . . . . . 9
Types of Long-Term Memory 10
Long-Term Memory in Frameworks 11
Technologies and Solutions for Advanced Memory Management 12
Enhancing Memory Accuracy with Named-Entity Recognition 13
Summary 15

**3. Some Economics of Agents, Model Usage, and Selection** . . . . . 17
The Economics of Agent Adoption 17
The Model Selection Matrix: A Multifactor Analysis 19
The Rise of the Multimodel Strategy 21
A Framework for Empirical Evaluation 22

**4. Navigating Agent Trade-Offs: Custom Builds, Frameworks, and Hosted Solutions. . . . . 25**
The Developer's Dilemma: Framework Versus Custom Architecture 25
Designing for the Future: Preventing Architectural and Vendor Lock-In 30

**5. Collective Memory: How Teams and Organizations Share Knowledge Through AI Agents. . . . . 33**
From Individual to Collective Intelligence 34
Team Knowledge Sharing Platforms and Tactics 35
Building Organizational Memory with AI Agents 36
Human-AI Team Collaboration Patterns 38
Looking Forward: The Feedback Loop 38

**Conclusion. . . . . 41**

# Introduction

Have you ever had a conversation with a manager that, however pleasant it started, devolved into frustration because they simply couldn't remember a critical detail from last week? Maybe you spent an hour explaining a new project, your concerns about the timeline, the specific approaches you'd decided on—only to have them ask a week later, "Wait, what are you working on again?" It's all you can do not to walk away in exasperation.

For those of us working with AI agents, this scenario feels painfully familiar. The agent you're collaborating with today might be brilliant at understanding your current request, even maintaining perfect context throughout a long conversation. But mention that project from last week—the one you meticulously outlined, with all its requirements and constraints—and you're met with the digital equivalent of a blank stare.

This isn't just an inconvenience; it's a fundamental limitation that shapes how we work with these systems. New sessions can feel like starting from scratch, forcing you to reexplain context that should be remembered and rebuild understanding that should persist. The agent's amnesia forces us into a loop of recontextualization—turning what should be an ongoing collaboration into a series of disconnected encounters.

The fact is that agentic memory is simply not as expansive as it should be. While these systems can process vast amounts of information within a single conversation, their ability to retain and meaningfully recall that information across time remains frustratingly limited.

The good news is that this limitation hasn't gone unnoticed. Researchers and engineers are working tirelessly to expand memory capacity, improve storage and retrieval mechanisms, and create systems that can maintain context not just for hours, but for weeks and months. They're tackling the problem from multiple angles—from fundamental improvements to attention mechanisms to clever workarounds that simulate longer-term memory.

This book is about understanding those efforts and, more importantly, what they mean for how we'll work with agents in the future. Because the organization or individual who figures out how to give their agents effective long-term memory won't just have a better tool—they'll have a true collaborator.

## Agent Memory: It's Just Data…Until It Isn't

At its core, agent memory is exactly what any software engineer would expect: data, storage, and retrieval. When you hear "memory management" for AI agents, you'd be right to think "data management." We have decades of experience with databases, caching systems, and storage architectures. So why do we need entire books about agent memory?

The answer is deceptively simple: while data storage may be a familiar topic, how agents *use* that data is fundamentally different from any system that engineers have built before.

Consider a traditional database. When you query for customer records, you get exactly those records, every time. The database doesn't decide that yesterday's query about customer #12345 is less important today. It doesn't compress old transactions into vague summaries. It certainly doesn't retrieve different results based on how you phrase your query.

But agents? They're nondeterministic by nature. The same query might pull different information based on subtle changes in phrasing. What gets stored isn't just data—it's embedded into vectors where *bank* (financial) and *bank* (riverside) might live in completely different neighborhoods of meaning. Retrieval isn't a precise `SELECT` statement; it's a fuzzy search through semantic space where relevance is calculated, not guaranteed.

This is why we borrow so heavily from human memory as a metaphor. Like humans, agents must work within constraints: limited

context windows instead of infinite storage. Agents weigh recent information more heavily than old. They make decisions about what to retain, what to compress, and what to forget. They retrieve information based on similarity and context, not perfect matches.

Yet even this comparison has limits. As we'll explore, agents don't learn continuously like humans do. They can't update their core knowledge. They're frozen in time, relying on clever engineering to simulate the kind of dynamic memory we take for granted.

This tension—between data management as we know it and the strange new requirements of nondeterministic systems—defines everything about how we build agent memory. It's why a simple conversation can become a complex dance of embeddings, vector databases, importance scoring, and semantic caching. It's why giving an agent the same task twice may yield different results, even with identical memories.

## What You'll Get from This Book

This book takes you on a journey from fundamental decisions to practical implementation, always keeping memory at the center of the conversation. Here's what each chapter delivers:

*Chapter 1*
: The technical foundation of this report, this chapter introduces agent memory management. Agents are non-deterministic by nature, so how can we teach them to retain and use important user information? Using strategies such as importance scoring, cascading memory systems, and checkpointing, we build conceptual foundations for the rest of the report.

*Chapter 2*
: Provides a deep dive into agent memory systems. This is the technical heart of the book, discussing the difference between episodic, semantic, and procedural memory. How agent memory actually works—from context windows to persistence strategies, importance scoring to semantic caching. This chapter threads the needle from theory to industry practice, covering common agentic frameworks like Redis and LangGraph.

*Chapter 3*
: Introduces the economics of agents, model usage, and selection. Memory isn't free. Every token stored costs money. Learn why

the economics of memory management keep shifting and what that means for your architecture.

*Chapter 4*
: Covers navigating agent tradeoffs—custom builds, frameworks, and hosted solutions. Should you build from scratch, adopt a framework, or use a hosted solution? Your choice shapes everything about how you'll manage memory.

*Chapter 5*
: Covers collective memory—how teams and organizations share knowledge through AI agents. This chapter spans individual agents to organizational intelligence. It covers how Transactive Memory Systems work in practice, why novices benefit most from shared knowledge, and how platforms like Zep and MCP connect organizational memory.

To close things out, we'll cover the future of agent memory. In the conclusion, you'll learn why memory is so impactful—and what it means for organizations to get agentic memory right.

Throughout this book, we'll return to a fundamental truth: the most important agent will always be the human agent. We are the conductors guiding these powerful orchestras. The systems we'll explore—from vector databases to semantic caching, from importance scoring to collective memory platforms—are instruments in our hands. They can store vast amounts of data, retrieve it in clever ways, even make intelligent decisions about what to keep and what to forget. But it's we who decide what constitutes success, what memories matter most, and how these systems should serve our goals.

This book will teach you not just what's important in agent memory management but why it's important. Because in the end, the organizations and individuals who thrive won't be those with the biggest context windows or the fastest retrieval algorithms. They'll be those who understand how to conduct these systems—instructing them in just the right ways to retain what matters, forget what doesn't, and transform raw data into genuine intelligence. Let's begin.

CHAPTER 1

# A Deep Dive into Agent Memory Systems

An agent's *memory* is, at its core, synonymous with data, storage, and retrieval. Anytime you hear "memory management," you would be right to interpret this as "data management." And data management, it turns out, is a concept we have a great deal of knowledge about in the world of software engineering. So why is it then that we need entire reports on memory management in AI agents? The answer, it turns out, is that while data management is an understood entity, the *usage* of data for AI agents is fundamentally different from anything that tool engineers have encountered before.

Recall that agents are nondeterministic systems, programmed with the ability to use tools and constraints that generally guide them—but nondeterministic all the same. To these agents, some data is more relevant than others, all data takes up space, and all agents have a limited amount of space—or context windows—in which to process. This fundamental insight shapes everything about how we design, implement, and manage agent memory systems.

Because the way in which agents use data differs fundamentally from traditional software programs, memory is often used as a stand-in for human cognition. The thinking is that if agents use information to generate nondeterministically—weighing recent information more heavily than older information, taking into account user context and preferences, and adapting to new inputs

dynamically—then perhaps memory is the appropriate analogy for how agents work.

This tension between humanlike behavior and fundamentally different architectures defines the current state of agent memory systems. In this chapter, we'll explore how the industry is navigating these challenges through short-term memory management, long-term persistence strategies, emerging technologies, and enhancement techniques like named entity recognition.

## Understanding Agent Memory Systems

Not every system will be the same for agents, and they are certainly going to change in the future—they're changing as we speak! That's why focusing on specific architectures is much less important than understanding higher concepts. Although agents aren't classically programmable, they follow classic computer science principles when it comes to memory management.

Think of agent memory as like RAM in a computer. Even as context windows expand, it's generally true that the more applicable and concise the information, and the more direct the query, the better the results will be. This isn't just about efficiency; it's about the fundamental nature of language paired with the systems that process this information.

As there are different architectures for agent systems, there are also different classifications of memory. Some distinguish between sensory memory (ingesting information like images, audio, and haptic feedback), short-term or working memory (an active memory buffer of conversation history), and long-term memory (storage relevant to the agent's or user's life or work).[1] Others focus on splitting between short-term and long-term memory, with subcategories for long-term memory including episodic (specific past events), procedural (contextual working knowledge and learned skills), and semantic (general world knowledge).[2] Much of this thinking is influenced by the field of psychology, which can categorize human

1 Michael Lanham, *AI Agents in Action* (Manning Publications, 2024), 200.

2 Manvinder Singh and Andrew Brookins, "Build Smarter AI Agents: Manage Short-Term and Long-Term Memory with Redis," Redis Blog, April 29, 2025, *https://redis.io/blog/build-smarter-ai-agents-manage-short-term-and-long-term-memory-with-redis/*.

memory similarly. These distinctions help tailor how agents interact with humans, with systems, and even with other agents. In particular, they influence how memory is stored and retrieved.

## Memory Storage and Representation

Almost all memory is embedded into vectors of continuous numbers meaningful to specific large language models (LLMs) and then stored in vector databases. Some of this information can be considered knowledge (embedded documents or working context), while other information is memory in the traditional sense (user preferences, standing instructions, or relevant past answers). Then there are episodic memories that don't need to be kept, such as random interactions or questions that don't appear to have lasting relevance.

The process of deciding what constitutes each type is where the complexity begins. Unlike traditional databases where you explicitly define schemas and relationships, agent memory systems must make these determinations dynamically, often with imperfect information.

## The Challenge of Nondeterministic Systems

Those of us who work with agents every day know the frustration: instructions explicitly given in the past that are no longer retained in a new session or, worse, instructions neglected during a longer session with the same agent. But even if memories are retained, that doesn't mean the agent will act as you'd expect. Because agents are not programmable in the traditional sense, giving one the same task twice may yield markedly different results.

Consider a research assistant that pulls different sources for the same question. This variability extends to coding agents, too. After all, asking a coding agent to scan and iteratively improve a large legacy codebase is a far more difficult task than asking one to build a new repository from scratch. Instructing an agent to "alter the search bar" in a system that could have many search bars leaves significant room for interpretation. What agents retrieve for you largely depends on factors such as the size of the context the agent must ingest, a complex interplay of their embeddings, their similarity metrics, and the specific phrasing of your query.

## Storage and Retrieval: The Core Challenge

This bears repeating because it is the most crucial aspect of memory management in AI agent systems: the retention of knowledge is dynamic and stochastic, not just on the storage side but on the retrieval side as well.

How do you decide what should be stored? What instructions are comprehensive enough to give a system? Do you store everything? Conversations can range from a few sentences to dozens of pages of text, depending on the user and use case. When storage gets tight, how do you flush the system?

There are many strategies to address these challenges. Some popular methods include:

*Importance scoring*
: Calculating memory importance based on recency, frequency of reference, user engagement metrics, and keyword relevance[3]

*Cascading memory systems*
: Allowing the agent itself to choose what to promote to long-term storage and what to retrieve[4]

*Intelligent compression*
: Using specialized models to condense conversation history into key details, events, and decisions[5]

*Vector store offloading*
: Moving older messages from short-term memory into vector stores, often with summarization[6]

Engineers typically compress information by instructing an LLM to summarize to the best of its ability. But summaries are not the same thing as the original. There is, by definition, loss of information.

3 Singh and Brookins, "Build Smarter AI Agents."

4 "Memory for Agents," LangChain Blog, October 19, 2024, *https://blog.langchain.com/memory-for-agents*.

5 "How to Migrate to LangGraph Memory," LangChain Documentation, *https://python.langchain.com/docs/versions/migrating_memory*.

6 Singh and Brookins, "Build Smarter AI Agents."

## The Imprecision of Retrieval

Retrieval makes everything more complicated. Information retrieval is often based on similarity between texts, and because language is imprecise, retrieval will be fuzzy, too. The classic example is *bank*: it can be a financial institution or an aspect of a river.

Depending on the model choice and embeddings, different algorithms must be used: cosine similarity, Euclidean distance, or even older methods like term frequency–inverse document frequency (TF-IDF). There's no single way to search and retrieve information, with plenty of trade-offs between speed and accuracy.

We're certainly improving in the vector database space—local options like ChromaDB, Redis, PostgreSQL with pgvector, and Qdrant exist—but there's still plenty of room for further improvement.

## Managing Context Window Limitations

All models have context windows containing information provided as context for generating answers. There are different methods for managing these limitations. With FIFO (first in, first out), the earliest information received over a long conversation may be least accurately recalled, meaning that more recent information gets prioritized.

Strategies to address this include intelligent pruning, where a model removes superfluous information. But there are consequences to this. Consider summarized information about a legal text: you might get the broad strokes of the legal argument and topics but lose critical information, like a negation or case reference that completely changes the content. Summarization by definition means losing detail and taking a higher abstract perspective.

Compare this to larger context windows like those in Gemini 2.5, which can handle millions of tokens. We can stuff more information into a model, but we may not have effective recall for the first-passed information relative to the last. It'd be nice if we lived in a world where models had perfect recall, but the architecture of transformers' self-attention mechanisms requires quadratically more processing as context increases.

Algorithms like FlashAttention attempt to work around this, but a more direct approach might be retrieval-augmented generation (RAG), which limits the corpus of documents and forces the LLM to return sourced information rather than stuffing superfluous information inside.

An even more refined approach—one I believe will become more popular—is semantic caching. What if we retained the relative context of an information retrieval system over time by processing the semantics of the content being passed? Frequently retrieved information gets prioritized. For systems like internal LLMs or RAG where many users talk to the same corpus of information, it may be both more computationally effective and cost-effective to semantically cache that information. Semantic caching isn't without its drawbacks—it works exceptionally well for single-shot questions but breaks down in multiturn conversations. These methods will continue to be refined, but the fundamental problems will remain the same: how can we most efficiently and effectively store and retrieve the information we desire?

## Persistence via Checkpointing

Checkpointing is a crucial step for agents, especially those that engage in multiturn conversations. As agents interact with a user or a system, they periodically save their internal states (their memory) in order to persist information across sessions or long conversations. Different organizations, products, and systems approach this process differently. The key insight is that checkpointing isn't just about saving state—it's about making that state retrievable and actionable in the dynamic, nondeterministic world of agent interactions.

There are many ways to handle checkpointing, and different teams use different tools. For example, Redis is a popular choice because it's fast and works well for real-time applications. Some setups use Redis to save conversation threads or agent state, making it easy to restore context across sessions or even share memory between different parts of a system. Features like automatic cleanup (time to

live) help keep things tidy, so you're not stuck with a bunch of old, irrelevant data.[7]

Basically, checkpointing is about making sure agents don't lose their place and that their memory is both persistent and practical in the unpredictable world of AI interactions.

7 Brian Sam-Bodden, "LangGraph & Redis: Build Smarter AI Agents with Memory & Persistence," Redis Blog, March 28, 2025, *https://redis.io/blog/langgraph-redis-build-smarter-ai-agents-with-memory-persistence/*; Redis Official Documentation, Redis, *https://redis.io/docs/*.

CHAPTER 2

# Long-Term Memory: Building Persistent Learning Agents

There's no single, universal definition for the different types of memory in agent systems—every company seems to have its own take. For example, Anthropic, OpenAI, and Google all use slightly different terminology and approaches. But the more interesting question is: how does an agent actually decide what counts as procedural, semantic, or episodic memory? Or put another way: what gets treated as short-term, long-term, or contextual memory?

Along these lines, semantic caching might play a big role. Sometimes short-term memories can be promoted to long-term if they're accessed frequently enough. Conversely, long-term memories that aren't used much might get summarized, become less detailed, or even be dropped from the system altogether. These are the kinds of trade-offs and decisions that go into designing agent memory. Ultimately, it all comes down to how the system is built to manage and retain information. Since it's nearly impossible to program an agent to handle every scenario, we rely on constraints and parameters to guide what gets treated as episodic, semantic, or procedural memory.

## Types of Long-Term Memory

The industry has converged on three primary types of long-term memory, although implementations vary significantly:

*Episodic memory*
: Stores specific past experiences and events, functioning like human autobiographical memory. Companies typically implement this through RAG systems on conversation histories, extracting relevant chunks instead of keeping the full history.[1] The common approach uses few-shot example prompting where agents learn from past sequences, with key events, actions, and outcomes logged in structured formats.[2]

*Semantic memory*
: Maintains structured factual knowledge—facts, definitions, rules—implemented through knowledge bases, symbolic AI, or vector embeddings. LLMs extract information from conversations, storing it as user or entity profiles that are retrieved and inserted into system prompts to influence future responses.

*Procedural memory*
: The least common area, but a growing one, which stores skills, rules, and learned behaviors for automatic task performance. This combines LLM weights, agent code, and system prompts, with some agents updating their own prompts through "reflection" or metaprompting.

The field of agentic AI is moving away from rigid definitions of memory toward more flexible hybrid approaches, where memories can transition between types based on usage patterns and importance scoring. As Rowan Trollope, CEO of Redis, observes, this mirrors human memory consolidation. Just as our REM cycle compresses information from short-term to long-term memory as we sleep, the goal is to build agents to engage in similar processes.[3]

1 Cole Stryker, "What Is AI Agent Memory?" IBM, accessed September 8, 2025, *https://www.ibm.com/think/topics/ai-agent-memory*.

2 "Memory: Agent Development Kit," Google ADK Documentation, *https://google.github.io/adk-docs/sessions/memory*.

3 Rowan Trollope, personal interview, July 11, 2025.

## Long-Term Memory in Frameworks

The practical implementation of long-term memory varies dramatically across frameworks, with each taking a different philosophical approach to the challenge:

*LangGraph Stores*
: From the creators of LangChain, a framework for LLM and agent development, this system organizes memory in namespaces as JSON documents with unique identifiers, supporting semantic facts, user preferences, episodic examples, and procedural system prompts. The LangMem SDK adds tools for extracting information from conversations, optimizing prompts, and maintaining persistent memory.[4]

*Mem0 (Memory-Zero)*
: This framework extracts key facts from interactions and updates long-term memory selectively. Rather than storing complete chat histories, it maintains concise entries to reduce memory usage and improve retrieval speed. Users can add a graph-based extension, Mem0g, that maps relationships between entities for additional context.[5]

*Redis Semantic Caching (LangCache)*
: This framework addresses the repetitive nature of agent queries through semantic caching. The implementation includes configurable search criteria, a REST API, and user-specific security features. At the time of writing, LangCache is in private preview, but it should soon be available to the general public.[6]

---

4 "Launching Long-Term Memory Support in LangGraph," LangChain Blog, October 8, 2024, *https://blog.langchain.com/launching-long-term-memory-support-in-langgraph/*; "LangGraph Memory Concepts," LangChain Documentation, *https://langchain-ai.github.io/langgraph/concepts/memory/*; "LangMem SDK for Agent Long-Term Memory," LangChain Blog, February 18, 2025, *https://blog.langchain.com/langmem-sdk-launch/*.

5 Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," arXiv:2504.19413, 2025.

6 Rowan Trollope, "Introducing LangCache and Vector Sets, Simple Solutions for High-Performing AI Apps," Redis Blog, April 8, 2025, *https://redis.io/blog/spring-release-2025/*; Jim Allen Wallace, "Semantic Caching for Faster, Smarter LLM Apps," Redis Blog, July 9, 2024, *https://redis.io/blog/what-is-semantic-caching/*; Rowan Trollope, personal interview, July 11, 2025.

*ADK MemoryService (Google's Agent Development Kit)*
: This framework provides a BaseMemoryService interface with two primary functions: adding completed sessions to storage and searching stored information. Developers can choose between InMemoryService for RAM-based keyword searches (nonpersistent) or VertexAIMemoryBankService for production environments with persistent semantic search.

## Technologies and Solutions for Advanced Memory Management

The landscape of agent memory technologies reflects a fundamental tension: the need for both sophistication and simplicity, performance and flexibility, innovation and reliability. Each solution represents a different philosophy about how agents should remember.

Each framework reflects different priorities and philosophies:

*LangGraph*
: Seeks simplicity with its document store approach. Easy integration and namespace organization make it ideal for rapid prototyping and standard agent workflows, though it lacks the sophistication of specialized solutions.

*ADK MemoryService*
: Provides enterprise-grade reliability with clear interfaces and Vertex AI integration. The trade-off is its limitation to the Google ecosystem—perfect for Google Cloud deployments but less flexible for multicloud architectures.

*Vector database alternatives*
: Each serves varying levels of demand. Pinecone offers managed services that are excellent for scale. Weaviate provides open source with hybrid search capabilities. Qdrant focuses on performance with advanced filtering. Chroma remains lightweight and developer friendly. Pgvector leverages familiar PostgreSQL tooling for teams already invested in that ecosystem. And Redis leverages its ubiquity as a caching system by adding RediSearch capabilities for indexing and search.

The choice of a vector database isn't simple (Figure 2-1). Do you need the raw performance of Redis? The intelligent abstraction of

Mem0? The simplicity of LangGraph? The enterprise features of ADK? Your answer shapes your agent's memory architecture and your overall system design.

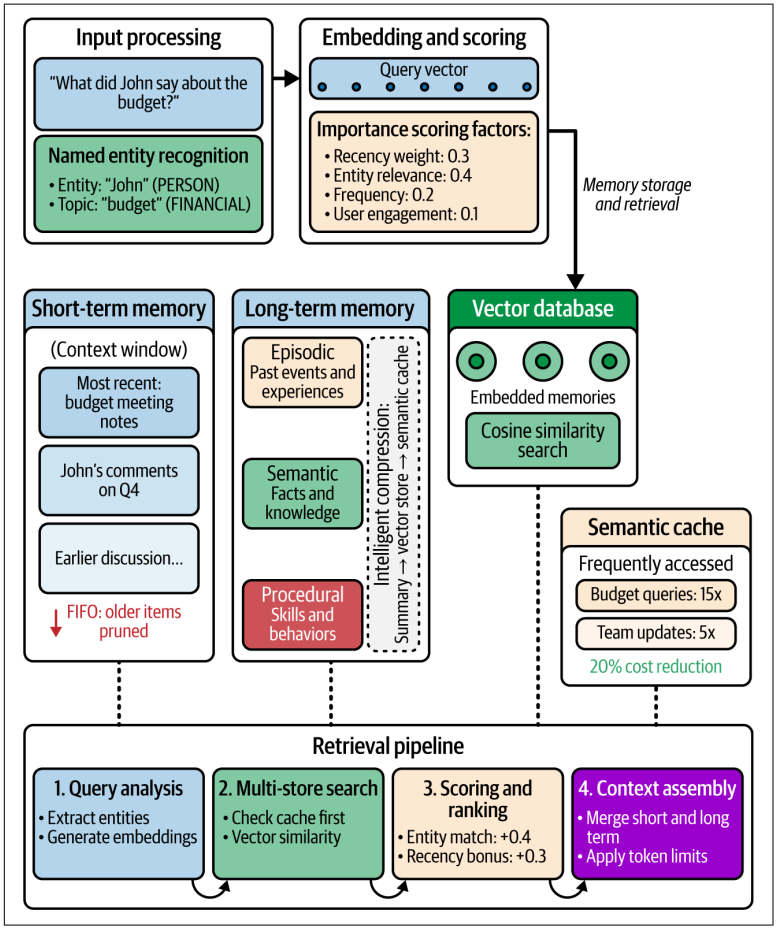


*Figure 2-1. Scoring and retrieval outline*

# Enhancing Memory Accuracy with Named-Entity Recognition

Named-entity recognition (NER) transforms the fuzzy world of natural language into the structured precision that computers—and agents—can reliably work with. In agent memory systems, NER isn't just a nice-to-have feature; it's becoming essential for accurate, retrievable memory.

## The NER Pipeline and Its Role in Memory

NER is a technique that helps computers pick out and label important things—such as people, places, organizations, or dates—from plain text. Think of it as a way for an agent to read a sentence and automatically highlight the names, companies, or locations mentioned, turning messy language into structured data.

Why does this matter for agents? When an agent can reliably identify and tag entities in conversations or documents, it can organize and retrieve information much more effectively. For example, an agent might use NER to keep track of which people were mentioned in a meeting, pull up all previous discussions about a specific project, or link related facts across different conversations. This structured approach makes memory retrieval more accurate and allows agents to answer questions like "What did John say about the budget?" or "When was the last time we discussed Paris?" with much greater precision.

Modern NER systems are impressively accurate, especially when fine-tuned for conversational AI.[7] Some agents go even further, using NER-enhanced pipelines to store entities as metadata, combine semantic and entity-based search, and maintain relationships between entities over time. This turns fuzzy, unstructured memory into something agents can more easily reason about and use.

NER can undergo the following three phases:

*Entity extraction*
: Identifies people, locations, organizations, dates, and custom entities, tagging each with confidence scores and linking entities across conversation turns

*Memory storage enhancement*
: Stores entities as structured metadata alongside embeddings, enables hybrid search combining semantic and entity-based approaches, and creates entity-centric memory indexes

7 Imed Keraghel, Stanislas Morbieu, and Mohamed Nadif, "Recent Advances in Named Entity Recognition: A Comprehensive Survey and Comparative Study," arXiv:2401.10825, 2024.

*Retrieval improvement*
: Allows for querying memories by specific entities, filtering results by entity type, and maintaining entity relationships over time

## Improving Memory Retrieval and Accuracy with Structured Data

NER isn't just about labeling words—it's about making memory search smarter and more precise. For example, if you ask an agent, "What did John say about the budget?" NER lets the agent filter its memory for both the person (John) and the topic (budget) instead of just doing a broad keyword search. This can shrink the search space dramatically and make answers more relevant.[8]

Beyond simple filtering, NER helps agents build knowledge graphs—connecting people, places, and things and tracking how they relate over time. This reduces confusion (such as distinguishing "Apple" the company from "apple" the fruit) and helps agents resolve pronouns or ambiguous references.

In practice, many production systems already use NER for better memory: Redis Agent Memory Server, Mem0g, and LangChain all include entity recognition or entity memory modules. As the field evolves, NER has begun to expand to handle not just text but also images and audio as well as to adapt to new domains with minimal training.

Structured data from NER turns vague, fuzzy memory into something agents can actually use and reason about—making them more helpful and accurate in real-world scenarios.

# Summary

Before moving into economic considerations, let's recap what we've learned about memory so far.

Agent memory is the process of processing, interpreting, storing, and retrieving data dynamically. This simple statement, which opened our exploration, has big implications for how we build and

8 "Entity-Based Memory Retrieval in Agent Systems," in *Proceedings of AAAI Conference on Artificial Intelligence*, 2024.

think about AI agents. Throughout this chapter, we've seen how this idea shapes everything about agent memory systems—from the way we draw inspiration from human cognition to the nuts and bolts of real-world implementations.

The real challenge isn't just technical: it's about embracing the unpredictability and flexibility that agents require. Agents aren't just databases with a fancy interface; they're dynamic, adaptive systems that need to balance remembering, forgetting, and learning on the fly.

If there's one takeaway, it's that building agent memory isn't about stuffing more data into bigger context windows or chasing the latest database. It's about designing systems that can manage information intelligently—knowing what to keep, what to compress, and what to let go. The future of agent memory will be shaped by how well we can give our agents the ability to adapt, prioritize, and make sense of the information they encounter.

In the end, memory is just data—but when managed well, it becomes knowledge, and when applied dynamically, it becomes intelligence. That's the real promise of agent memory systems: turning raw data into something agents and humans can actually use, one interaction at a time.

CHAPTER 3

# Some Economics of Agents, Model Usage, and Selection

LLMs serve as the cognitive engines that power AI agents—just like how different car engines are built for different purposes, from ATVs to school buses. Extending this framework, outlined in *What Are AI Agents? When and How to Use LLM Agents* (O'Reilly), selecting the right LLM for your agent isn't just about performance—it's also about a trade-off between capabilities and economics. The foundation model you choose will directly affect not only what your agent can do but also how much it costs to run, how efficiently it operates, and ultimately, whether your project makes financial sense. Understanding these economic trade-offs is crucial for anyone building AI agents, whether you're a startup watching every dollar or an enterprise looking to scale.

## The Economics of Agent Adoption

The first thing to note about AI agents is that as they expand in their capacity to access and work with more tools—through protocols like Model Context Protocol (MCP) or through functional calls within LLMs themselves—the end user benefits more from the agent. But as with everything in economics, there is a trade-off. As the complexity of the task an agent is provided increases, so too does the marginal cost of a given action—whether we measure that cost by the hours it takes to complete a task, the cost of API calls, or the ability of the underlying agent to solve the niche tasks.

The chart shown in Figure 3-1 plots task complexity (*x*-axis) against marginal benefit and marginal cost (*y*-axis). As complexity rises, the extra value an AI model adds (marginal benefit) falls while the extra cost or risk (marginal cost) climbs. Their intersection (*) marks the economically optimal point: any task with complexity to the left of * still delivers a positive net benefit (MB > MC), making AI deployment worthwhile; tasks to the right cost more than they're worth.

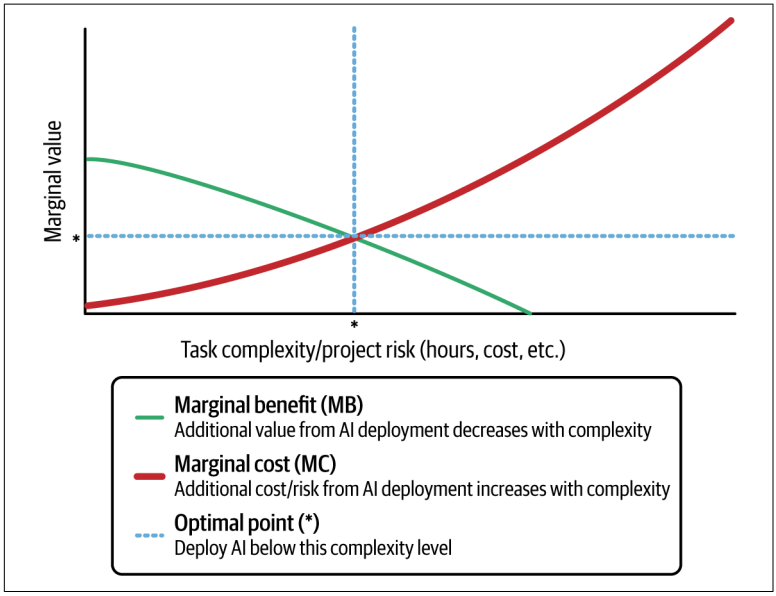


*Figure 3-1. Current AI capabilities*

But there's more. When breakthrough models or new integration capabilities arrive—think GPT-5 or a cutting-edge open source foundation model—the marginal cost curve shifts downward and to the right. In practical terms, each additional unit of task complexity now carries a lower cost, so the intersection with the marginal benefit curve moves farther right. That expands the zone where MB > MC, meaning more complex tasks become economically viable for AI automation. In other words, as model efficiency improves, your "deployment frontier" widens—AI can add positive net benefit to a broader range of projects (Figure 3-2).

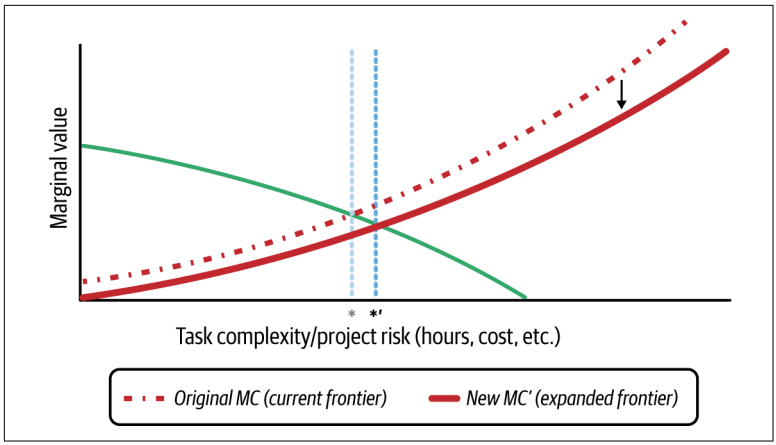


*Figure 3-2. When new AI models emerge, the frontier shifts*

# The Model Selection Matrix: A Multifactor Analysis

Building an AI agent requires a corresponding LLM to run it. The choice of an LLM is not as simple as it may seem. Defaulting to the latest model, such as Claude Sonnet 4 or GPT-5, may be a solid heuristic for quick, general projects, but when it comes to specialized applications or applications that may become the core of your business or product, much more thought, planning, and explanation are required. Do you only need text-based agents, or do you need multimodal agents that can ingest voice or vision inputs? How many users will use your agents? What are your latency needs? These questions are just table stakes.

Making these decisions effectively requires understanding how different models perform across several key dimensions. Some factors—like quality and accuracy—seem obvious, but others, like the relationship between context window size and cost, can catch teams off guard when they reach production scale.

## Core Performance Metrics

When it comes to AI agents, there's a general trade-off between speed and reasoning agents' output and reasoning. Many in the industry have started distinguishing between what they call "System 1" versus "System 2" thinking—borrowing theory of the

human mind developed in the parallel domain of behavioral economics by Daniel Kahneman and Amos Tversky.[1] System 1 processes are fast, automatic, and instinctual: the mental heuristics and quick judgments we use to navigate daily life without expending too much cognitive energy. System 2 processes are deliberate and require conscious effort, the kind of rational deliberation and critical thinking we associate with complex problem solving.

### Quality and accuracy

In LLMs, techniques like chain-of-thought prompting essentially force the model into System 2 mode, where it reexamines its logic iteratively. This reasoning process can significantly improve accuracy and reliability, but it can come at a cost: more processing time and higher compute expenses. The trade-off is real—System 2 thinking can deliver better results but costs more to run.

The good news? Clever engineering strategies can sometimes recover most of the quality gains of chain-of-thought processing, with two to three times less latency or token usage.[2] Techniques like early exit inference and speculative decoding are making it possible to get closer to System 2 quality without the full System 2 cost.

### Cost

The most well-known LLMs (OpenAI's GPTs, Claude's Sonnet, Google's Gemini) provide proprietary APIs for pay-as-you-go usage. Prices vary by model and are constantly being revised based on the market and the latest model, but do *not* let the cheap per-token cost of most models fool you. The average project usually requires the user to pass in the context of their code, writing, or documents to prime the model—accruing context windows in the hundreds of thousands—if not millions—of tokens *per day*. These costs can quickly add up. Figure 3-3 shows the cost per million tokens of 21

1 Daniel Kahneman, *Thinking, Fast and Slow* (Farrar, Straus, and Giroux, 2011).

2 Yaniv Leviathan, Matan Kalman, and Yossi Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, 2023; Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, et al., "LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 12622–12642*, 2024.

LLMs from some of the most popular providers. As the size and quality of the model increase, so too do the prices.

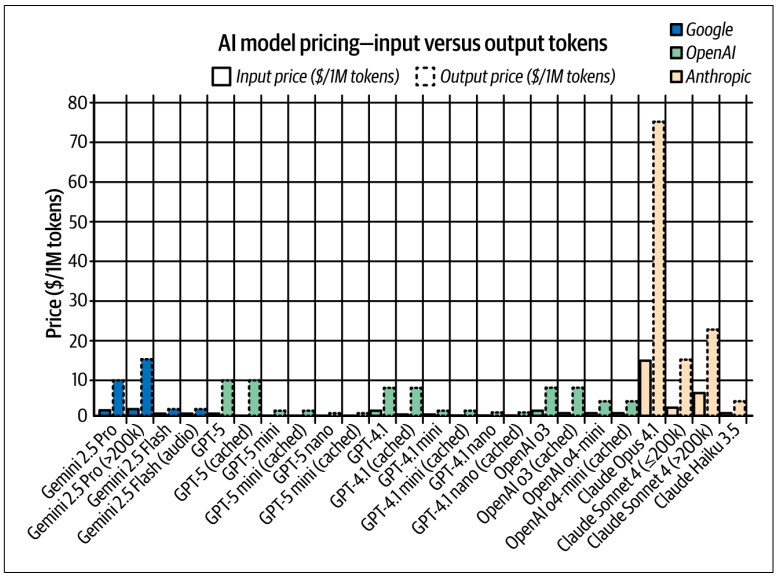


*Figure 3-3. AI model pricing comparison*

In fact, there is a significant discrepancy between the cost of content you provide a model and the cost of content it provides you. Ingress costs among LLMs are relatively cheap, but the egress costs from the LLM back to you can be substantial. The more you ask the model to return, the more your costs will rise. So for those companies looking to position themselves as central to their product, it bears careful consideration of how much these models will be used and how much you can afford to spend.

# The Rise of the Multimodel Strategy

As agentic systems become more complex, organizations and industry experts are increasingly abandoning a one-size-fits-all approach to model selection. Instead, a more sophisticated multimodel strategy is emerging as a best practice. This involves using a combination of different LLMs rather than sticking to a single LLM vendor. In this multimodel strategy, model selection is context dependent and optimized to the specific requirements of a particular task within a larger workflow:

*Specialized roles for different models*

A common and effective pattern is to create a hierarchy of models. A highly capable but more expensive and potentially slower model, such as o3 Pro or Claude Opus 4, is used for the most complex cognitive tasks, such as high-level planning, complex reasoning, or decomposing a difficult problem into subtasks. Once the plan is formulated, the execution of the individual, more straightforward subtasks is delegated to a fleet of smaller, faster, and more cost-effective models, such as Claude Sonnet 4, Gemini Flash 2.5, or GPT-5 mini. This approach allows the system to achieve a balance between reasoning and cost-efficient execution.

*Scalable orchestration*

For large enterprises deploying hundreds or even thousands of agents, relying on a single, powerful, monolithic model for every operation can be prohibitively expensive and difficult to scale. A more scalable architecture may involve using sparse models, such as mixture-of-experts (MoE) models like Mixtral, or orchestrating a large number of specialized lightweight agents. This allows the system to scale cost-effectively while maintaining high performance.

# A Framework for Empirical Evaluation

In such a fast-changing industry, many look to industry benchmarks as useful signals for which models they should use. While public benchmarks such as these are a useful starting point for comparing models, they cannot guarantee how a model will perform on your specific project or proprietary use case. It is therefore essential for all agent and LLM developers and architects to develop their own domain-specific evaluation frameworks to test models against their unique data and requirements. Here's how.

## Creating Use-Case-Specific Test Criteria

The first step to developing an internal evaluation framework is to define clear, objective, and measurable criteria for what constitutes success for the agent's task—after all, you cannot manage what you cannot measure. These evaluation criteria should go beyond simply checking for a single "right answer" and should instead assess

the quality of the entire agentic process. Key areas for evaluation include:

*Task completion*
: The most fundamental metric: did the agent successfully achieve its high-level goal? Specifically, did a piece of substantive work get completed in a timely and relatively correct manner?

*Tool correctness and efficiency*
: Did the agent select the correct tools for the job? Did it invoke them with the correct parameters? Was the tool usage efficient, or were there redundant or unnecessary calls?

*Reasoning coherence and relevance*
: Was the agent's internal chain of thought logical? Did its reasoning steps directly contribute to solving the problem, or were they irrelevant?

## The "LLM-as-a-Judge" Methodology

Take it from a practitioner: evaluating thousands of agent outputs is impractical and does not scale. A powerful technique that has emerged to address this challenge is the "LLM-as-a-judge" approach. This method involves using a powerful, state-of-the-art LLM to act as an impartial evaluator. The judge LLM is given the agent's output, the original prompt, a set of ground-truth data, and a detailed evaluation rubric or scoring template. It is then prompted to assess the agent's performance against the defined criteria and provide a score and a qualitative explanation for its judgment. To improve the reliability of this method, it is common to use advanced prompting techniques like chain-of-thought prompting, which instructs the judge LLM to think step-by-step through its evaluation, making its reasoning more transparent and less prone to random variability. This methodology enables the scalable, repeatable, and nuanced evaluation of agent performance, although it requires careful prompt engineering and awareness of potential biases in the judge model itself.

CHAPTER 4

# Navigating Agent Trade-Offs: Custom Builds, Frameworks, and Hosted Solutions

*Digital organizations shift the primary focus of integration from the systems to the capabilities, emphasizing clean interfaces over those capabilities.*

—Brandon Byars

Whether you are an insurance company or a startup, the fundamental considerations at the start of the process of deploying an AI agent are largely the same: build versus buy, frameworks versus customization, and of course, cost and scalability concerns. If these trade-offs and considerations sound familiar, they should. Despite AI agents being a wholly new category of technology, their *fundamentals* are those of most software products. That should give us heart because we have decades of precedent upon which we can rely to build resilient solutions.

## The Developer's Dilemma: Framework Versus Custom Architecture

Not all projects involving or centered around AI agents require the same tooling considerations. Ultimately, the choice between, say, using LangGraph to build out an agent and turning to a hosted service such as Glean AI comes down to one thing: scope. Is the project you're about to undertake existential to your company's

future, or is it experimental and exploratory? How much data will you need to ingest and process regularly? Will your initial user base begin as large or small? Is this purely an internal tool, or will it be customer facing? Does your company already have a robust infrastructure supporting AI agents, or is this project the first of its kind? All of these questions and more must be, if not answered, at least understood in order to reasonably select a proper AI agent architecture for your project.

Figure 4-1 shows a helpful decision framework for considering some of the most common questions that arise when considering the build-versus-buy decision for AI agents.

**AI agent architecture decision grid**

| Use case scenario | Custom build | Framework | Hosted solution |
|---|---|---|---|
| Internal productivity tool for small teams (<50 users) | Learning opportunity | Consider | Recommended |
| Consumer-facing AI assistant with unique domain knowledge | Best fit | Good option | Vendor dependent |
| Rapid prototype for executive demo (2–4 weeks) | Risky timeline | Tight timeline | Ideal |
| Enterprise solution with strict compliance (HIPAA, SOC2) | Maximum control | With care | Vendor dependent |
| Research/experimental project with uncertain requirements | Beware of trade-offs | Flexible | Low commitment |
| High-volume production system (millions of interactions) | Performance | Beware of limitations | Beware of costs |

*Figure 4-1. AI agent architecture decision framework*

But a simple chart only scratches the surface of the complexities involved. The following three sections break down the case for build, the case for frameworks, and the case for hosted solutions respectively.

## The Case for Build

Whether to build from scratch is likely the most consequential decision you can make when starting an AI agent project. It demands significant resources, both in labor and capital. However, it offers unmatched control, and if agents are core to your business, there's no better decision you can make. Here are a few considerations:

*"If it's a core business function—do it yourself no matter what."*

This time-tested principle from Joel Spolsky's classic essay on build-versus-buy decisions is especially relevant for AI agents. When AI becomes core to your competitive advantage, frameworks and hosted solutions can turn into strategic liabilities by limiting your ability to differentiate and iterate rapidly.[1]

*Unparalleled flexibility and customization*

When you build AI agents from scratch, you're limited only by the ability of your engineers to code the system you desire and the availability of the integrations you hope to access. This empowers unique and specific solutions to your business problems and software systems that most frameworks will be unable to match. Consider this: the more bespoke software systems you have in place, the less likely an open source framework will meet all your needs.

*The AI agent becomes more valuable as it becomes more extensible*

The entire point of the AI agent revolution isn't simply that AI agents are dynamic chatbots—it's that their dynamism extends to their ability to call and access tools. But as Brandon Byars notes in his essay "You Can't Buy Integration," we shouldn't think of this as merely "connecting systems together, or sharing data to keep systems in sync." The real goal is to create "clean interfaces between capabilities" that drive organizational agility. For AI agents, this distinction is foundational to the agility they enable organizations to achieve. When you build from scratch, you have complete control over how these interfaces are designed and optimized. As Byars states, success should be measured "by increasing digital agility over time," where "those digital capabilities become the primary value driver, arguably even more important than the systems themselves."[2]

*Performance and cost optimization at scale*

While not every company should build its own foundational models or infrastructure, there's a compelling economic case

1 Joel Spolsky, "In Defense of Not-Invented-Here Syndrome," Joel on Software, October 14, 2001, *https://www.joelonsoftware.com/2001/10/14/in-defense-of-not-invented-here-syndrome*.

2 Brandon Byars, "You Can't Buy Integration," MartinFowler.com, December 14, 2021, *https://martinfowler.com/articles/cant-buy-integration.html*.

for custom AI agents as usage scales. External services typically charge per API call or token, meaning costs grow linearly (or worse) with success. Custom builds flip this equation: after the initial development investment, increased usage often reduces per-interaction costs through economies of scale. Your infrastructure costs become more predictable and controllable, and performance can be optimized specifically for your use patterns rather than generic workloads. For companies expecting high-volume AI agent interactions, this cost-structure advantage can become substantial over time.

## The Case for Frameworks

Using frameworks to build agents is often the best starting point for new projects. If you're new to agents, frameworks provide guidance through up-to-date documentation, state-of-the-art capabilities, and fast iteration. If you're more experienced, frameworks still help you build quickly and gather feedback for proof-of-concept work. Here's a breakdown of their main benefits:

*Accelerated development and prototyping*

When you are building an AI agent, start with the fundamentals: what is the project scope? If scope remains unclear but stakeholder demos are critical, consider established frameworks like LangGraph, AutoGen, and CrewAI, which have quick tutorials on building agents zero-to-one on their websites. Remember, you can always refactor code from a framework to a customization. But it's much more difficult to refactor the other way around—from a customization to a framework.

*Lower barrier to entry*

The greatest benefit of a good framework is its lower barrier to entry—that's the whole point of an abstraction. What's easier: building a car from scratch or buying one premade? The same logic applies to AI agents. If your team is new to agents, it's much wiser to leverage a prewritten open source framework that thousands of people have tried, tested, and kicked the tires on.

*Encapsulation of best practices*

So long as you are careful, frameworks will be the encapsulation of industry best practices. In the best-case scenario, adopting a modern, popular framework will put you in an advantageous

position to leverage proven patterns and tools. But be careful in your choice of frameworks. Always scope out the potential use cases for your AI agent. Ask any seasoned developer about the pains of having chosen (or been forced into) the wrong framework—suddenly, a simple integration turns into weeks of work that could have been written manually in hours.

## The Case for Hosted Solutions

Hosted solutions are arguably the most straightforward cases to consider. If your organization lacks robust experience or infrastructure to build AI agents, they're usually a safe bet:

*Fastest time to value*
: When it comes to zero-to-one deployments of an AI system, you're usually not going to get faster results than a hosted solution. Hosted platforms such as Glean or Cognigy focus solely on providing customers with the ability to deploy agents with minimal overhead. Cloud providers such as Amazon Web Services (AWS), Google Cloud, and Azure all offer their own versions of agent platforms that, while typically requiring more intensive work, will more easily integrate into existing enterprise cloud networks.

*No infrastructure overhead*
: The primary value of hosted solutions is the lack of infrastructure management required. For every hour spent managing infrastructure, users could be focusing on core tasks that drive business value instead.

*Professional support and service-level agreements (SLAs)*
: Most hosted solutions include professional support and SLAs that severely derisk the reliance on such tools, especially by enterprise staff who have less experience with generative AI and machine learning engineering.

*Continuous updates and improvements*
: These solutions regularly enhance their features and product offerings, soliciting feedback from their user base to maintain feature parity with the competition and to retain best practices.

# Designing for the Future: Preventing Architectural and Vendor Lock-In

While the allure of rapid deployment and expansive capabilities makes AI vendor solutions attractive, understanding how these systems create dependencies is crucial for maintaining long-term flexibility and control over your technology stack.

## The Nature of AI Lock-In

There are many manifestations and flavors of vendor lock-in. Here are three of the most common:

*Proprietary model APIs*
: Some API designs require you to code around certain frameworks and specifications that are vendor and framework specific and are not adopted industrywide. Avoid this if you can. The only appropriate time is if you are in a niche industry or require niche data that simply cannot be returned another way.

*Nonexportable fine-tuned models*
: Some vendors provide the option to fine-tune models on their systems but do not provide the option to export their model weights in hopes you will simply use their APIs. This can come up when resource-constrained companies look for quick fixes but can leave your IP held hostage.

*Integrated data and memory systems*
: By far the most consequential lock-in occurs in the storage of critical data on external AI vendor systems—data such as conversation histories, user metadata, and vector embeddings for memory. Migrating this data can become costly because of substantial egress fees, technical challenges, and even the inability to host that data on your own systems.

These lock-in phenomena are not mutually exclusive, nor are they collectively exhaustive. Only by planning ahead can you mitigate these risks effectively.

## Architectural Strategies for Portability

Beyond understanding lock-in risks, successful organizations implement concrete architectural patterns that preserve flexibility while still leveraging the power of modern AI services:

*Modularity and abstraction*

The most effective strategy to avoid vendor lock-in (and overcomplexity) is to design agent systems with modular, composable architectures. This enables easy swapping between different vendors or even between external vendors and nascent and growing internal services that could eventually replace the vendors altogether. The easiest way to achieve this is to build abstractions around vendor APIs such that the internal API remains consistent and agnostic to the external API call. Such an abstraction can serve as, say, a generative AI (GenAI) chat-completion service that is able to route its calls dynamically between GPT-5, Sonnet 4, and an internally hosted model. If your organization later decided to switch models from Anthropic to, say, Google Gemini, or even to abandon external LLMs altogether, only the internal GenAI service would need to be updated, while the core application code would remain unchanged.

*Open standards*

Whenever possible, reduce dependency risks by building on well-established standards and protocols rather than proprietary ones. This ensures greater interoperability between your systems and vendors. This does not mean you should integrate the bleeding edge of open source tooling—the field of GenAI is too tumultuous to lock yourself in. For example, while MCP has garnered a great deal of interest, there's no guarantee it will become the industry norm. Balance innovation with stability.

*Containerization*

A tried-and-true method, containerization is one of the key architectural decisions a team can make to ensure flexibility and portability of their app—flexibility that can be used to avoid vendor lock-in. When we containerize an app, we encapsulate our entire application and its dependencies into a single portable unit. This container can then be deployed on almost any cloud platform (AWS, Google Cloud, Azure, etc.) or even on premises for those companies who anticipate their agents

becoming central to their business. This approach maximizes deployment options and effectively decouples an agent application from its underlying host environment.

By leaning on the fundamentals of software engineering, teams can more confidently navigate the build-versus-buy decision space for AI agents. The key is to anchor decisions in the function of the agent and its strategic relevance to your business. If your goal is to run a quick proof of concept, a hosted solution offers a fast and low-risk path to user feedback. If you're experimenting with new paradigms, agent frameworks provide a flexible and low-friction starting point. But if your company is committed to riding the agent wave for the long haul, it's essential to start with a framework, iterate quickly, and then graduate to your own custom stack—one that protects your independence and positions you for long-term agility and control.

CHAPTER 5

# Collective Memory: How Teams and Organizations Share Knowledge Through AI Agents

So far, we've focused on individual agents in AI systems: how they store, retrieve, and manage information. But why limit agent memory to a single user's interactions? Organizations and teams constantly build contextual stores of knowledge while executives, managers, and team leads struggle to effectively collect, store, and disseminate this information to those who need it.

AI agents offer a solution: shared memory systems that preserve organizational knowledge beyond any individual's tenure. With agents, the possibility emerges of a binding organizational memory that persists beyond any one person—dampening the impact of retirements, promotions, and role changes that naturally erase critical institutional insights. But how exactly can this process work, and what does this mean for the future of work?

## From Individual to Collective Intelligence

Let's start by clarifying what we mean by *collective organizational knowledge*. Psychologists have a useful framework for this: transactive memory systems (TMSs). A *TMS* is defined as the "group-level knowledge sharing and memory system for encoding, storing, and retrieving information from different knowledge areas in a group."[1] In essence, it's about "knowing what other team members know" and being able to access that knowledge when needed. This helps assemble the different pieces of distributed group knowledge into one coherent "group mind"—and this group mind is directly associated with team effectiveness.

Rather than simply storing individual knowledge with an AI agent, new architectures are being developed to expand from one-on-one agent-human interactions to broader agent-human-team interactions with shared knowledge bases. The data stores are exactly the kind of TMSs that hold the potential to improve team effectiveness (Figure 5-1). Indeed, there's convincing evidence that such centralized systems of knowledge benefit not only existing team members but newly onboarded and novice employees as well. In a recent 2023 call center study, AI assistants deployed to support customer interactions demonstrated this effect dramatically: novice workers improved their productivity by 34% while experienced workers saw minimal gains, suggesting that the AI effectively captured and disseminated the expertise of top performers throughout the organization.[2]

1 Nadine Bienefeld, Michaela Kolbe, Giovanni Camen, Dominic Huser, and Philipp Karl Buehler, "Human-AI Teaming: Leveraging Transactive Memory and Speaking Up for Enhanced Team Effectiveness," *Frontiers in Psychology* 14 (2023): 1208019, *https://doi.org/10.3389/fpsyg.2023.1208019*.

2 Erik Brynjolfsson, Danielle Li, and Lindsey R. Raymond, "Generative AI at Work," NBER Working Paper No. 31161 (April 2023), *https://doi.org/10.3386/w31161*.

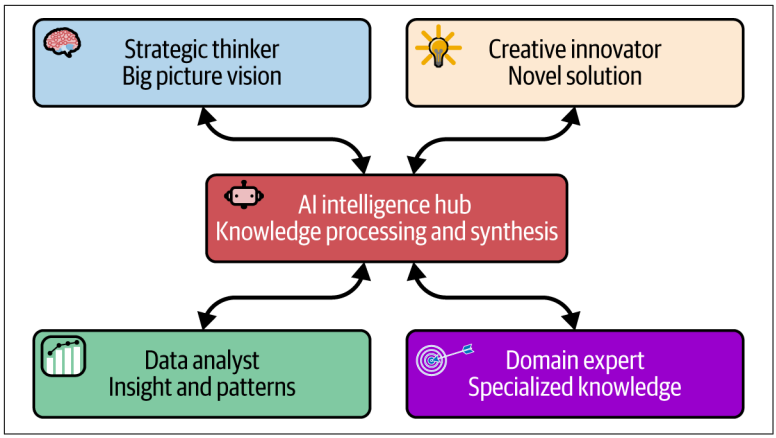


*Figure 5-1. Example of AI integration in a TMS*

The shift from siloed knowledge to shared organizational memory represents more than a technical evolution. When a senior engineer's problem-solving approach can be captured and made accessible to junior team members through AI agents, we're not just preserving information—we're democratizing expertise.

## Team Knowledge Sharing Platforms and Tactics

So what options are there for developing one's own internal knowledge system or TMS? The landscape of AI-powered knowledge management platforms reveals sophisticated approaches to team collaboration that go far beyond simple document storage.

Several platforms demonstrate different approaches to building organizational memory:

*Zep*

A memory platform that builds "temporal knowledge graphs" from team interactions and business data, creating memory systems that track how information changes over time. Zep maintains intraorganizational information across all team interactions—remembering preferences, conversations, and business knowledge—which enables AI agents to access collective team expertise and transfer best practices from top performers to newer members, similar to the knowledge-dissemination effects in the call center study.

*Onyx*

An open source AI platform that connects enterprise applications (Google Drive, Slack, Confluence, Salesforce, etc.) to create a unified knowledge search and AI assistant system. The platform enables teams to find shared information while preserving security. With custom AI assistants that can be embedded directly into existing workflows, Onyx builds searchable institutional memory stores that can scale from small teams to thousands of users.

*MCP*

Although MCP is not strictly a platform but rather is a protocol, it standardizes how applications provide context to LLMs. MCP enables teams to build persistent memory systems through servers like the Knowledge Graph Memory Server, which maintains entities, relationships, and observations across conversations. Unlike traditional platforms, MCP creates a decentralized approach where multiple lightweight servers can expose specific capabilities—from local files to remote APIs—while maintaining a unified interface for AI agents. This architecture allows teams to construct custom knowledge graphs that evolve with each interaction, storing not just information but also the relationships and context between different pieces of organizational knowledge.[3]

## Building Organizational Memory with AI Agents

Having a choice in which platform you adopt is helpful—but how are these tools capturing the critical organizational knowledge in the first place? Once again, the answer comes down to novel storage and retrieval strategies.

3 "Introducing the Model Context Protocol," Anthropic, November 25, 2024, *https://www.anthropic.com/news/model-context-protocol*.

## Capturing Institutional Knowledge

Across these platforms, several common strategies emerge for building organizational memory:

*Continuous learning from daily work*
: AI agents can observe and learn from senior experts' problem-solving approaches through their daily interactions, capturing not just what they do but also how they approach problems.

*Contextual knowledge preservation*
: Unlike traditional documentation methods, AI agents can preserve the context around decisions: why certain choices were made, what alternatives were considered, and what constraints existed at the time.

*Dynamic knowledge graphs*
: Modern systems like those implementing the A-MEM (agentic memory) framework create interconnected knowledge networks following the Zettelkasten method. When new memory is added, the system generates comprehensive notes with contextual descriptions, keywords, and tags, creating a web of related information.[4]

## Memory-Preservation Strategies

Organizations are implementing several strategies to ensure knowledge persistence across agent lifecycles:

- Checkpointing mechanisms that periodically save agent state and learned patterns
- Hierarchical memory systems with short-term, long-term, and archival storage tiers
- Cross-agent knowledge synchronization to ensure that discoveries by one agent benefit the entire system
- Version control for agent memory, allowing rollback and historical analysis

4 Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang, "A-MEM: Agentic Memory for LLM Agents," arXiv:2502.12110, 2025. *https://doi.org/10.48550/arXiv.2502.12110*.

## Human-AI Team Collaboration Patterns

For all this talk about memory patterns, storage, and retrieval, we've overlooked the most crucial element of AI agent collaboration: the humans themselves. If organizations hope to successfully integrate agents into their teams, they must create the space, opportunity, and trust necessary for employees to understand how and why these agents will enhance—not replace—their work.

The integration of AI agents into team memory systems requires significant organizational change. The requirements for workforce transformation include:

*Employee empowerment*
: There is no better way to encourage AI use than to give employees the keys to their tools. Asking which agents employees want to use and giving them the freedom to experiment is a surefire way to increase adoption.

*Cultural shift to augmentation*
: Some employees will be rightfully skeptical of new tooling. Rather than dismissing or ignoring these feelings, organizations should clearly establish how agents are not a "replacement" for employees but are cognitive partners that amplify human capabilities rather than substitute for them.

*Trust-building through transparency*
: Establishing clear guidelines about what AI agents can access, how they learn, and what decisions remain solely human based builds trust between organization leaders who wish to encourage AI use and employees who want control over their own choices.

## Looking Forward: The Feedback Loop

When we started this chapter, we viewed agentic memory as something between a single human and an AI agent. By now, it should be clear that's only one piece of a much larger system. As more employees interact with organizational agents, these systems capture and store increasing amounts of context. This stored context then becomes accessible to other team members, transforming previously tacit knowledge into shared resources that boost productivity across the organization. We're witnessing the beginning of a powerful

feedback loop: the more people use these systems, the smarter and more valuable the systems become. As organizations adopt enterprise-level knowledge stores, this dynamic will only accelerate. The companies that thrive will be those that successfully transform their AI agents from mere tools into collaborative team members, their static data into living knowledge, and their individual expertise into collective intelligence.

# Conclusion

As stated throughout this book, you cannot predict the future. We're just at the starting line of the AI era. But if there were a prediction to make, the smart money isn't on tooling integration—it's on memory.

Tools remain relatively static: once you've integrated with them, you generally retain access. Memory, on the other hand, is dynamic. A project that was a high priority one week may become history the next. The best agents will integrate this changing context and proactively expand critical memory and expunge extraneous information.

This dynamic nature of memory is why the current moment presents such a unique opportunity. Unlike traditional software where expertise accumulates over years, there are no true experts in agents yet. The world of AI is simply too fast-paced and fluid for one person to be an expert in everything there is to know. This levels the playing field—the key to expertise for agents is exposure to these systems, to the use of agents, to what they can be used for successfully, and to how they can augment, not replace, your work.

For organizations looking to capitalize on this opportunity, experimentation is key. For some, that will look like giving users licenses and letting them experiment. For other, likely larger organizations, it may be wiser to slowly integrate agents together in seminars and hackathons. It's an equal playing field, and the only way to truly learn is by doing with agents. Through this collective exploration, teams naturally develop their own TMSs, learning not just what agents can do but also who knows what about using them effectively.

As these experiments unfold, organizations will quickly discover that memory management is the critical challenge. True, memory will always be finite—because memory is data, and data always

needs to be stored somewhere. We also know that, unless the attention mechanism that underlies LLMs fundamentally changes, the search space for LLMs will remain quadratic. That means that retrieval will be limited into the known future as well.

But it's not about having infinite memory—it's about *how* we retain and access memory. The algorithms we use, how we route between them based on context—these are the real differentiators. We can be confident that caching will play a growing role in agentic memory management. After all, think about search engines: when a question is asked—such as "What time are the playoffs today?"—it's very likely not the only instance of that question being asked. Smart search engines cache these queries to make them more readily available—so too will agents of the future.

The same principle applies to queries on knowledge bases within an organization. As we explored in Chapter 2, semantic caching retains the relative context of an information retrieval system over time by processing the semantics of the content being passed. Information that is retrieved frequently gets prioritized. For systems like internal LLMs or RAG where many users talk to the same corpus of information, it's both more computationally effective and cost effective to semantically cache that information.

This is where the real innovation is happening. Researchers aren't just fighting the quadratic-attention problem with bigger context windows; they're getting smarter about importance scoring—calculating memory importance based on recency, frequency of reference, user-engagement metrics, and keyword relevance. They're building cascading memory systems that allow the agent itself to choose what to promote to long-term storage. This adaptive approach—knowing what to keep, what to compress, and what to let go—is the key to scalable, dynamic memory systems moving forward.

The organizations that get memory right, that prioritize shared memory systems, that understand these trade-offs of memory retention and retrieval will be the organizations that have an edge in productivity. After all, if memory equals data, then those with the best systems for managing that data will understand not just what is, but what can be.

Yet in all this discussion of systems and algorithms, we must remember a crucial truth: the most important agent will always be

the human agent. We are the conductors guiding the machine. Like an orchestra playing Beethoven's Fifth, the music may be known, the notes memorized, but it's the conductor who knows exactly what they want from this rendition—the conductor who has done the research, who has an overarching vision. In the same way, even if agents begin to write code for us, do our research, communicate to others for us—it will be us, the humans, who will decide on what we want, who will guide the agent not only on what success is but also what *we* want success to be.

While we cannot know what the future will bring, we can anticipate that the users of agents will play a bigger role in personalizing how they "conduct"—instructing them in just the right ways to retain important information. The future isn't about perfect recall or infinite memory. It's about designing systems that can manage information intelligently, guided by human judgment and values. With guidance from classic software engineering principles and research into how to better leverage our new stochastic systems, we're only just getting started.

## About the Authors

**Benjamin Labaschin** is currently a principal machine learning engineer and founding hire at Workhelix. Prior to this role, he worked as a senior data scientist at Hopper, XPO Logistics, and Blackstone. He holds a background in economics and environmental science. His areas of expertise include machine learning, MLOps, DevOps, and backend engineering. Ben has a particular passion for scaling engineering systems to support the evolving needs of hyper-growth startups and for advancing sustainable engineering practices.

**Jim Allen Wallace** is group manager of product marketing at Redis, where he bridges technical know-how and market insights to help developers apply Redis in the age of AI. He's worn hats as an engineer, salesperson, and entrepreneur, giving him a front-row seat to how products get built, sold, and loved (or ignored). When he's not shaping go-to-market strategy, Jim can be found hiking the mountains outside Denver, Colorado, or maybe pushing his cat Chutney off his keyboard.

**Andrew Brookins** is a principal applied AI engineer at Redis, where he builds open source infrastructure for memory, retrieval, and intelligent agents. He created the Agent Memory Server and Redis OM Python, and contributes to Redis libraries and applied AI tooling. With a background in backend systems and engineering leadership, he focuses on tools that help developers build smarter, more reliable systems.

**Manvinder Singh** is VP of AI products at Redis, where he is responsible for the portfolio of AI offerings, including vector search, semantic caching, and agent memory. Previously, he spent over 10 years in various AI and cloud roles at Google. As a director of product management for AI ecosystem at Google, he was responsible for orchestrating and driving product efforts to engage with and build with the ecosystem across their AI portfolio, including the Gemma family of models.